

Sesión de clase 07 · Práctica: Transformers locales desde Jupyter Notebook

Los modelos de la Parte 1 ven el texto como una **bolsa de palabras**: el orden se pierde. Un **Transformer** asigna a cada token un vector que depende de todos los demás (autoatención), así distingue «no llegó tarde» de «llegó tarde».

Todo corre **local, en CPU**: cada modelo se descarga una sola vez (cientos de MB) a la caché de Hugging Face y después funciona sin internet.

Modelo	Tarea	Salida
cardiffnlp/twitter-xlm-roberta-base-sentiment	sentimiento	negative · neutral · positive
MoritzLaurer/mDeBERTa-v3-base-mnli-xnli	zero-shot con NLI	etiquetas propias

Secciones: 1. El orden importa · 2. `pipeline` de sentimiento · 3. Reseñas de entrega · 4. Zero-shot con NLI · Discusión.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import cohen_kappa_score, f1_score
from transformers import pipeline

pd.set_option("display.max_colwidth", 120)
```

1. Bolsa de palabras: el orden cambia el significado

Dos opiniones opuestas con exactamente las mismas palabras.

```
A = "El envío no llegó tarde, llegó bien"
B = "El envío no llegó bien, llegó tarde"

bow = CountVectorizer()
X = bow.fit_transform([A, B])
vectores = pd.DataFrame(X.toarray(), columns=bow.get_feature_names_out(),
                        index=["A", "B"])
print("¿x_A == x_B?", (vectores.loc["A"] == vectores.loc["B"]).all())
vectores
```

¿x_A == x_B? True

	bien	el	envío	llegó	no	tarde
A	1	1	1	2	1	1
B	1	1	1	2	1	1

Como $x_A = x_B$, **cualquier** modelo lineal sobre conteos (lexicón, TF-IDF + regresión logística, Naive Bayes) les asigna el mismo puntaje.

2. pipeline de sentimiento

`pipeline` junta el **tokenizador**, el **modelo** y la conversión de la salida en etiquetas. La primera ejecución descarga el modelo.

```
clf = pipeline("text-classification",
               model="cardiffnlp/twitter-xlm-roberta-base-sentiment")
clf([A, B])
```

Loading weights: 100%  201/201 [00:00<00:00, 22331.40it/s]

```
[{'label': 'positive', 'score': 0.5884841084480286},
 {'label': 'negative', 'score': 0.7565773129463196}]
```

El Transformer sí separa A (positivo) de B (negativo): «tarde» precedido por «no» y «tarde» al final de la frase producen vectores distintos.

Tokenización por sub-palabras

Las palabras raras se dividen en piezas frecuentes; así el vocabulario es finito y no hay palabras desconocidas. El `_` marca el inicio de una palabra (tokenizador SentencePiece de XLM-RoBERTa).

```
tokens = clf.tokenizer.tokenize("no me pidieron datos innecesarios")
print(len(tokens), "tokens:", tokens)
print("Vocabulario del modelo:", clf.tokenizer.vocab_size, "piezas")
```

```
8 tokens: ['_no', '_me', '_pidi', 'eron', '_datos', '_inne', 'ces', 'arios']
Vocabulario del modelo: 250002 piezas
```

¿Dónde falla una regla de palabras?

Los cuatro comentarios de la pregunta inicial de la clase. La regla «si contiene *no* o una palabra negativa → negativo» los marca a todos como negativos.

```
ejemplos = pd.DataFrame({
    "caso": ["A", "B", "C", "D"],
    "estrellas": [4, 4, 3, 2],
    "texto": [
        "El registro de cuenta fue rápido y no me pidieron datos innecesarios.",
        "El checkout como invitado es rápido si no quieres crear una cuenta.",
        "El diseño del sitio es agradable pero en modo oscuro algunos textos son difíciles de leer.",
        "Las fotos de un producto no coincidían del todo con el color que llegó.",
        "No es un problema grave, pero genera cierta desconfianza.",
    ],
})
pred = clf(ejemplos["texto"].tolist())
ejemplos["modelo"] = [p["label"] for p in pred]
ejemplos["confianza"] = [round(p["score"], 2) for p in pred]
ejemplos
```

	caso	estrellas	texto	modelo	confianza
0	A	4	El registro de cuenta fue rápido y no me pidieron datos innecesarios.	neutral	0.56
1	B	4	El checkout como invitado es rápido si no quieres crear una cuenta.	neutral	0.75
2	C	3	El diseño del sitio es agradable pero en modo oscuro algunos textos son difíciles de leer.	negative	0.71
3	D	2	Las fotos de un producto no coincidían del todo con el color que llegó. No es un problema grave, pero genera cierta ...	negative	0.82

El modelo ya no marca A y B como negativos (los lee como neutros: no hay queja) y sí detecta la insatisfacción de D. En C el contraste con «pero» pesa más que el elogio, igual que en la regla del lexicón; el cliente puso 3★.

3. Reseñas de entrega

- `batch_size=16` procesa varios textos a la vez.
- `truncation=True` corta textos más largos que el máximo del modelo (512 tokens).
- `label` es la clase elegida y `score` la confianza del modelo en esa clase (0 a 1).

```
r = pd.read_csv("../data/resenas_entrega.csv")

pred = clf(r["texto"].tolist(), batch_size=16, truncation=True)
r["sentimiento"] = [p["label"] for p in pred]
r["confianza"] = [round(p["score"], 2) for p in pred]
r[["calificacion", "sentimiento", "confianza", "texto"]].head(8)
```

	calificacion	sentimiento	confianza	texto
0	1	negative	0.76	Recibi el pedido incompleto, faltaba el cable de carga original del OnePlus 12 dentro de la caja. Tuve que esperar u...
1	3	positive	0.53	El segundo cargador Belkin llego sin contratiempos, en el tiempo estimado y correctamente empacado.
2	5	positive	0.91	La segunda Switch Lite que regale llego perfecta y justo a tiempo para el cumpleaños que la necesitaba. Toda la expe...
3	5	positive	0.77	El segundo television TCL llego perfecto, bien empacado y sin ningun dano, muy distinto a la primera experiencia neg...
4	4	neutral	0.66	Me confirmaron por correo la garantia de un ano del cable Anker sin ningun problema.
5	5	neutral	0.51	Los audifonos Bose se emparejaron al instante con mi telefono, cero complicaciones.
6	2	negative	0.76	La instalacion de este segundo pedido de Family Hub se retraso otra vez, casi diez dias despues de la entrega del eq...
7	3	positive	0.65	La segunda camara instantanea llego completa y a tiempo, sin ningun inconveniente en esta ocasion.

¿Coincide con las estrellas?

Llevamos ambas cosas a tres clases: estrellas 1-2 → **neg**, 3 → **neu**, 4-5 → **pos**; y las etiquetas del modelo **negative/neutral/positive** a las mismas abreviaturas.

```
a3 = lambda s: "neg" if s <= 2 else ("neu" if s == 3 else "pos")
r["y"] = r["calificacion"].map(a3)
r["pred"] = r["sentimiento"].str[:3]

print(f"macro-F1 = {f1_score(r['y'], r['pred'], average='macro'):.3f}")
print(f"kappa = {cohen_kappa_score(r['y'], r['pred']):.3f}")
pd.crosstab(r["y"], r["pred"], rownames=["estrellas"], colnames=["modelo"])
```

macro-F1 = 0.552

kappa = 0.350

modelo neg neu pos

estrellas

	neg	neu	pos
neg	18	6	1
neu	9	9	3
pos	1	27	36

Sin haber visto **ni un ejemplo** de la tienda, el modelo ya supera claramente al azar ($\kappa > 0$). Pero fue entrenado con **tuits**: muchas reseñas de 4-5★ escritas en tono descriptivo («llegó en la fecha estimada») las lee como **neutras**. El error más frecuente es positivo → neutro, no positivo → negativo.

Probabilidades de las tres clases

Con **top_k=None** el pipeline devuelve el puntaje de **todas** las clases. Con eso armamos un puntaje continuo: P(positivo) – P(negativo), de –1 a +1.

```
todas = clf(r["texto"].tolist(), batch_size=16, truncation=True, top_k=None)
probs = pd.DataFrame([d["label"]: d["score"] for d in fila for fila in todas])
r["puntaje"] = (probs["positive"] - probs["negative"]).round(2)

r.groupby("calificacion")["puntaje"].agg(["count", "mean"]).round(2)
```

	count	mean
calificacion		
1	10	-0.58
2	15	-0.39
3	21	-0.18
4	29	0.37
5	35	0.58

El puntaje medio sube con las estrellas: el modelo ordena bien los textos aunque sus clases no coincidan exactamente con las del cliente.

4. Zero-shot con NLI: etiquetas propias sin entrenar

Un modelo de **inferencia textual (NLI)** decide si un texto *implica* una frase. El pipeline `zero-shot-classification` arma una hipótesis por etiqueta con `hypothesis_template` y elige la más implicada. Sirve para **aspectos** («¿de qué habla?») y también para polaridad.

```
zs = pipeline("zero-shot-classification",
              model="MoritzLaurer/mDeBERTa-v3-base-mnli-xnli")

zs("El envío tardó diez días",
  candidate_labels=["entrega", "pago", "sitio web", "soporte"],
  hypothesis_template="Este comentario trata sobre {}.")
```

```
/Users/ferris/.virtualenvs/mineria-web-2026-2-sspb/lib/python3.14/site-
packages/torch/jit/_script.py:1485: FutureWarning: `torch.jit.script` is not
supported in Python 3.14+ and may break. Please switch to `torch.compile` or
`torch.export`.
  warnings.warn(
```

Loading weights: 100%  202/202 [00:00<00:00, 6083.24it/s]

```
{'sequence': 'El envío tardó diez días',
 'labels': ['entrega', 'soporte', 'sitio web', 'pago'],
 'scores': [0.9988775849342346,
            0.0005281950579956174,
            0.0004162076802458614,
            0.0001779628946678713]}
```

```
texto = "El envío internacional tardó más días de los indicados originalmente."
res = zs(texto,
         candidate_labels=["satisfecho", "con una opinión neutral", "insatisfecho"],
         hypothesis_template="El cliente está {}.")
pd.DataFrame({"hipotesis": [f"El cliente está {l}." for l in res["labels"]],
              "probabilidad": [round(s, 2) for s in res["scores"]]})
```

	hipotesis	probabilidad
0	El cliente está insatisfecho.	0.49
1	El cliente está con una opinión neutral.	0.28
2	El cliente está satisfecho.	0.22

El cliente calificó este texto con **3★**: el modelo lo lee como insatisfacción. La etiqueta de referencia también tiene ruido.

Aspectos en las reseñas de entrega

```
ASPECTOS = ["tiempo de entrega", "estado del producto", "empaquete",
            "pedido incompleto", "atención al cliente"]

res = zs(r["texto"].tolist(), candidate_labels=ASPECTOS,
        hypothesis_template="Esta reseña trata sobre {}. ", batch_size=16)
```

```
r["aspecto"] = [x["labels"][0] for x in res]
r["aspecto_score"] = [round(x["scores"][0], 2) for x in res]

(r.groupby("aspecto")
 .agg(n=("texto", "size"), estrellas=("calificacion", "mean"),
      puntaje=("puntaje", "mean"))
 .sort_values("estrellas").round(2))
```

	n	estrellas	puntaje
aspecto			
pedido incompleto	5	2.20	-0.29
empaque	17	3.18	0.09
tiempo de entrega	26	3.38	0.04
estado del producto	53	3.87	0.23
atención al cliente	9	4.00	0.27

```
r.sort_values("aspecto_score", ascending=False).groupby("aspecto").head(1)[
 ["aspecto", "aspecto_score", "calificacion", "texto"]]
```

	aspecto	aspecto_score	calificacion	texto
87	estado del producto	1.00	4	El disco WD Blue no ha dado ningun problema desde que lo instale.
105	tiempo de entrega	1.00	2	El Samsung The Frame llego tres dias despues de la fecha estimada sin ninguna notificacion previa del retraso. Cuand...
24	atención al cliente	1.00	4	Resolvieron mi consulta sobre el hervidor Xiaomi mediante un video explicativo por correo.
65	empaque	0.99	4	Los JBL Tune 510BT llegaron bien empacados junto con el resto de mi pedido. El unico detalle menor fue que el manual...
61	pedido incompleto	0.80	3	El robot aspiradora llego completo aunque el manual venia en ingles a pesar de haberlo pedido en un sitio en espanol.

Discusión

- `cardiffnlp` fue entrenado con tuits y `nlptown` (Ejercicio 2) con reseñas de productos. ¿Cuál esperan que se adapte mejor a `resenas_entrega.csv` ?
- Cambien `hypothesis_template` por "El problema principal es {}." : ¿cambian los aspectos asignados?
- Escriban un comentario con ironía («Genial, tercera vez que me cobran doble») y pruébenlo en `clf` . ¿Lo detecta?

```
# Espacio para experimentar
clf(["Genial, tercera vez que me cobran doble",
     "Qué rapidez: solo tardaron tres semanas en responder mi reclamo"])
```

```
[{'label': 'positive', 'score': 0.798257052898407},  
 {'label': 'positive', 'score': 0.4163312613964081}]
```

La ironía sigue siendo difícil: el léxico positivo («genial», «rapidez») puede dominar aunque la intención sea negativa.